

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS

INTRODUCCIÓN A LA PROGRAMACIÓN Y COMPUTACIÓN 1

MANUAL TÉCNICO

Centro de Rescate Animal:
Gestión de Refugio y Adopciones

Estudiante	Yeremy Manuel Sánchez Ambrocio
Carné	202207466
Sección	F
Semestre	Segundo semestre 2026

Documento técnico de referencia para la estructura y funcionamiento interno del sistema.

1. Objetivo del documento

Este manual describe la estructura interna del sistema, la organización de sus clases, el manejo de arreglos estáticos y matrices, las reglas de negocio, la persistencia de datos, la generación de reportes HTML y el registro de bitácora. Su propósito es facilitar el mantenimiento del proyecto y la explicación técnica de las decisiones de implementación.

2. Arquitectura del proyecto

El proyecto está organizado por responsabilidades. La interfaz gráfica se encuentra separada de las clases que almacenan y procesan los datos. Las entidades representan los objetos del sistema; las clases de datos manejan arreglos estáticos y reglas de negocio; persistencia administra archivos; reportes genera HTML; util registra la bitácora y controlador gestiona la autenticación.

Paquete	Responsabilidad
modelo	Entidades: Animal, Adoptante, Solicitud, Rescate, EspacioRefugio, Usuario y Bitacora.
datos	Arreglos estáticos, búsquedas, registros, actualizaciones, eliminaciones y reglas de negocio.
persistencia	Carga y guardado de información en archivos de texto separados por punto y coma.
interfaz	Ventanas Java Swing y conexión entre controles gráficos y clases de datos.
reportes	Generación de los cuatro reportes HTML.
util	Registro y lectura de la bitácora.
controlador	Autenticación local mediante los usuarios almacenados en archivo.

3. Clases del modelo

Animal: representa un animal del refugio con código, nombre, especie y estado.

Adoptante: representa una persona interesada en adoptar, con DPI, nombre, teléfono, dirección y estado.

Solicitud: relaciona un adoptante con un animal y registra fecha y estado de la solicitud.

Rescate: almacena información de un caso de rescate, incluyendo fecha, ubicación, especie, descripción, prioridad y estado.

EspacioRefugio: representa una posición de la matriz de ubicaciones mediante fila, columna y código del animal asignado.

Usuario: almacena usuario, contraseña y rol.

Bitacora: representa un registro de fecha, usuario, acción y detalle.

4. Estructuras de datos

El proyecto utiliza arreglos estáticos de objetos, evitando estructuras dinámicas como ArrayList, HashMap, List, Queue o similares. Cada módulo tiene una capacidad máxima y utiliza un contador para controlar cuántos elementos están almacenados.

Módulo	Estructura	Capacidad
Animales	Animal[]	100
Adoptantes	Adoptante[]	100
Solicitudes	Solicitud[]	100
Rescates	Rescate[]	100
Usuarios	Usuario[]	100
Ubicaciones	EspacioRefugio[][]	4 x 5

El contador de cada clase permite insertar nuevos elementos en la siguiente posición libre y también recorrer únicamente la parte utilizada del arreglo.

5. Principales clases de datos y métodos

AnimalDatos: registrarAnimal(), listarAnimales(), buscarAnimal(), editarEstado(), eliminarAnimal(), coincideTexto(), getCantidad() y getAnimal().

AdoptanteDatos: registrarAdoptante(), listarAdoptantes(), buscarAdoptante(), buscarAdoptanteActivo(), editarAdoptante(), cambiarEstado(), eliminarAdoptante() y getCantidad().

SolicitudDatos: registrarSolicitud(), listarSolicitudes(), buscarSolicitud(), cambiarEstado(), atenderSolicitud() y getCantidad().

RescateDatos: registrarRescate(), listarRescates(), buscarRescate(), cambiarEstado(), cambiarPrioridad(), eliminarRescate() y getCantidad().

EspacioRefugioDatos: consultarEspacio(), asignarAnimal(), liberarEspacio(), buscarAnimal(), mostrarMatriz(), contarDisponibles(), contarOcupados(), contarOcupadosEnFila() y contarDisponiblesEnFila().

UsuarioDatos: registrarUsuario(), autenticación y consulta de usuarios.

6. Reglas de negocio y validaciones

Las operaciones se validan antes de modificar los arreglos. Se controlan campos vacíos, duplicidad de códigos o DPI, capacidad máxima, estados válidos y relaciones entre registros.

Área	Validación implementada
Animales	Código único; búsqueda por código/nombre/especie/estado; estados permitidos; eliminación lógica.
Adoptantes	DPI de 13 dígitos; DPI único; estados Activo/Inactivo/Eliminado.
Solicitudes	Adoptante activo; animal existente y no eliminado/adoptado; no duplicar solicitud activa; solo una solicitud por animal.
Rescates	Prioridad Alta/Media/Baja; transición controlada Registrado → En atención → Completado.
Ubicaciones	Área y espacio válidos; celda libre para asignar; animal existente y válido; animal no duplicado en matriz.

7. Persistencia de datos

La información se guarda en archivos de texto ubicados en la raíz del proyecto. Los campos se separan mediante el carácter ;. Cada clase de persistencia se encarga de guardar y cargar el arreglo correspondiente.

Archivo	Información
animales.txt	Registros de animales.
adoptantes.txt	Registros de adoptantes.
solicitudes.txt	Solicitudes de adopción.
rescates.txt	Casos de rescate.
espacios.txt	Fila, columna y código del animal de cada espacio.
usuarios.txt	Usuarios, contraseñas y roles.
bitacora.txt	Acciones registradas por el sistema.
estudiante.txt	Nombre, carné y sección del estudiante.

El proceso general es: iniciar ventana → crear estructura en memoria → cargar archivo → realizar operación → validar → actualizar datos → guardar archivo → actualizar la tabla/interfaz.

8. Matriz de ubicaciones

La matriz tiene 4 filas, correspondientes a las áreas Recepción, Cuarentena, Recuperación y Adopción, y 5 columnas correspondientes a los espacios 1 a 5. Cada celda contiene un código de animal cuando está ocupada o una cadena vacía cuando está disponible.

La asignación verifica que la posición esté dentro de los límites, que el animal exista, que no esté eliminado o adoptado, que el espacio esté libre y que el mismo animal no esté asignado en otra posición. La liberación elimina el código almacenado en la celda.

9. Autenticación y roles

La autenticación es local. Los usuarios se cargan desde **usuarios.txt** y se comparan las credenciales ingresadas con los registros almacenados. Los roles definidos son Administrador y Auxiliar.

10. Reportes HTML

La clase **ReporteHTML** genera cuatro documentos HTML: reporte de animales, reporte de adopciones, reporte de ocupación y reporte de bitácora. Los nombres incluyen fecha y hora para diferenciar cada generación.

Los reportes se construyen como cadenas HTML y contienen tablas con los datos correspondientes. Además, la generación de un reporte queda registrada en la bitácora.

11. Bitácora

La clase **BitacoraUtil** registra fecha, usuario, acción y detalle en **bitacora.txt**. También permite leer los registros para mostrarlos en el reporte HTML de bitácora. Se utiliza un arreglo fijo de cadenas para la lectura, manteniendo el enfoque del proyecto de no utilizar colecciones.

12. Interfaz gráfica

La interfaz está dividida en ventanas Swing. **VentanaLogin** controla el acceso; **VentanaPrincipal** funciona como menú; las demás ventanas corresponden a los módulos de animales, adoptantes, solicitudes, rescates, ubicaciones, reportes y datos del estudiante.

Cada ventana recibe las acciones del usuario mediante botones y campos, llama a las clases de datos para ejecutar las operaciones y actualiza las tablas de resultados. La lógica de almacenamiento y validación se mantiene en las clases de datos para evitar concentrar toda la lógica en la interfaz.

13. Flujo general de datos

Usuario → Interfaz Swing → Clase de datos → Validación → Arreglo/Matriz → Persistencia → Actualización de interfaz.

En operaciones relevantes también se registra la acción mediante **BitacoraUtil**. Los reportes consultan los datos almacenados y generan documentos HTML.

14. Estructura resumida del proyecto

```
src/centrorescateanimal/Main.java
src/controlador/LoginController.java
src/modelo/*.java src/datos/*.java
src/persistencia/*.java
src/interfaz/*.java
src/reportes/ReporteHTML.java
src/util/BitacoraUtil.java
```

15. Consideraciones de mantenimiento

Para agregar una nueva operación se recomienda mantener la separación de responsabilidades: la entidad debe representar los datos, la clase de datos debe contener la regla de negocio, la persistencia debe encargarse del archivo y la interfaz debe limitarse a capturar datos y mostrar resultados.

Antes de modificar una regla de estado o una relación entre módulos, se debe comprobar su efecto sobre los arreglos, la persistencia, la matriz de ubicaciones y los reportes. También se debe verificar que no se introduzcan colecciones dinámicas que contradigan las restricciones del proyecto.